

The Easier Way

Hooking Swift with a bridge
Håvard Sørbo



MAScon Berlin

October 2026



The Easier Way

frida-swift-bridge, rewritten

WHAT IT ADDS

- Generics: types, methods, and type arguments in hooks
- Closures: JavaScript functions where Swift wants one
- Async: call with `await`, hook through to completion
- Properties: hook getters, setters, and `modify` coroutines
- Calling convention: read from the binary, not guessed
- Lookup on demand: nothing is wrapped up front

WHERE IT RUNS

	arm64	arm64e	x86-64
macOS	✓	✓	✓
iOS	✓ [†]	✓ [†]	—
Linux	✓	—	✓
Windows	✓	—	✓

[†] device test suite, not in CI

Swift 5.7 → 6.4 1,000+ tests

github.com/hsorbo/frida-swift-bridge

agent.js: Two Types, Two Members, a Closure

```
7  const SymmetricKey = Swift.struct("CryptoKit.SymmetricKey");
8  const HKDF = Swift.struct("CryptoKit.HKDF");
9
10 const initData = SymmetricKey.$initializer("init(data:)");
11
12 const deriveKey = HKDF.$typeMethod("deriveKey(inputKeyMaterial:outputByteCount:)");
13
14 function bytesOf(value) {
15   let bytes;
16   const readAll = Swift.closure(buf => { bytes = buf.readBytes(); });
17   value.withUnsafeBytes(readAll);
18   return bytes;
19 }
```

agent.js

Types by qualified name. Metadata, not symbols: stripping doesn't hide types. HKDF is generic; the same line finds it.

Members by selector. `init(data:)` and `deriveKey(...)` are the names you'd write in Swift. No mangling.

A JS function where Swift wants a closure. `withUnsafeBytes` is called like a method; the buffer arrives as an object.

agent.js: Two Hooks

```
21  const base64 = bytes => Buffer.from(bytes).toString("base64");
22
23  Swift.Interceptor.attach(initData.address, {
24    onLeave(key) {
25      console.log(`SymmetricKey(data:) -> ${key.bitCount} bits: ${base64(bytesOf(key))}`);
26    },
27  });
28
29  Swift.Interceptor.attach(deriveKey.address, {
30    onEnter(args) {
31      console.log(`HKDF<${this.typeArguments[0]}>.deriveKey  outputByteCount=${args[1]}`);
32      console.log(`inputKeyMaterial: ${base64(bytesOf(args[0]))}`);
33    },
34    onLeave(key) {
35      console.log(`derived: ${base64(bytesOf(key))}`);
36    },
37  });
```

agent.js

The return value, decoded. `onLeave(key)` is the `SymmetricKey`: `key.bitCount` runs the getter. No x8, no saving it on entry.

Arguments by position. `args[0]` is the input key, `args[1]` the byte count, already a number.

The hash by name. `this.typeArguments[0]` is SHA384. The witness-table walk from the hard way, done for you.

agent.js: Async

```
40 // Async
41 Swift.Interceptor.attachAsync(Swift.function("__C.NSURLSession.data(from:delegate:)"), {
42     onEnter(args) {
43         console.log(`data(from: ${args[0].absoluteString})`);
44     },
45     onComplete(retval, error) {
46         if (error !== undefined) {
47             console.log(`threw ${error}`);
48             return;
49         }
50         const [data, response] = retval;
51         console.log(`${new ObjC.Object(response).statusCode()}, ${data.count} bytes`);
52     },
53 });
```

agent.js

Found by name, through an ObjC class.

NSURLSession is Objective-C; Foundation extends it in Swift. __C is Swift's own name for imported C and ObjC types.

onComplete, not onLeave. An async function returns at its first suspension. This fires when the response exists.

Result or error. The tuple (Data, URLResponse) as an array, or the thrown error, decoded.

Demo

TARGET.APP

SwiftUI, macOS, plain arm64. Two buttons.

Crypto runs the same `doCryptoStuff()` as Francesco's iOS app: the same password, `HKDF<SHA384>`, 80 bytes.

Fetch awaits `URLSession.shared.data(from: https://example.com/)`.

```
open Target/build/Target.app
frida -n Target -l agent.ts
```

EXPECTED

```
SymmetricKey(data:) -> 192 bits: aGVsbG8gaGFyZG9vZGVkIHBhc3N3b3Jk
HKDF<CryptoKit.SHA384>.deriveKey outputByteCount=80
inputKeyMaterial: aGVsbG8gaGFyZG9vZGVkIHBhc3N3b3Jk
derived: ZTZsR4IizkkEHsYYJByygs79dGQQ77TfVU5sQBht...
data(from: https://example.com/)
200, 577 bytes
```

The base64 of the input reads "hello hardcoded password": 24 bytes, 192 bits. The byte count is whatever example.com serves that day.

Finding Things

```
Swift.type("MyApp.Robot")           // or .class / .struct / .enum
Swift.struct("CryptoKit.HKDF<CryptoKit.SHA256>") // a specialization
Swift.modules.CryptoKit.SymmetricKey // lazy namespace per module

Swift.enumerateTypes("CryptoKit.*")   // glob over the qualified name
Swift.enumerateFunctions("*NSURLSession.data(*)") // hookable as they are
Swift.function("CryptoKit.HKDF.deriveKey(inputKeyMaterial:outputByteCount:)")
robot.greet                          // by bare name: address, signature, hook target

HKDF.$type.typeMethods()             // reflection: lists, builds nothing
HKDF.$type.specializations()         // every HKDF<H> built so far

Swift.choose(Swift.class("MyApp.Session")) // live instances
```

Types come from metadata. Every Swift image lists its types; stripping doesn't hide them.

Members are symbols. Exports first, then the symbol table, demangled and matched by selector. `robot.greet` is the member itself: address, signature, hookable.

Nothing up front. A lookup asks Frida's built-in Swift resolver. Listing is free: no type is built until you use it.

Bare names are Swift, \$ is the bridge. `robot.greet` is theirs; `robot.$type` is ours.

Calling

```
const robot = Swift.type("MyApp.Robot").init("R2");
robot.greet("hi"); // overloads by arity
robot.$method("pick()", { returnType: "Swift.Int" }) // by return type
robot.$method("echo", { typeArguments: [Int] }).call(7); // generic

Swift.type("Swift.Int")["*"](6, 7); // operators: 42
Swift.function("MyApp.Robot.greet(_:)").call(robot, "hi");
Swift.function(app, "$s5MyApp7computeY2iF").call(21) // mangled
const mightThrow = Swift.NativeFunction(addr, Int, [Int], { throws: true });
try { mightThrow(1); } catch (e) { e.value; } // the thrown error, decoded

using scratch = Swift.type("MyApp.Robot").init("S1"); // $copy, $dispose, using
robot.$field("name").write("C3"); // storage, no accessor
```

Call anything that has a name. The convention comes from the signature and from the type's own code, not from you.

Swift errors are JS exceptions. A throwing call lands in catch as a `SwiftError` with the thrown value decoded; typed throws(E) too.

Not in the binary, so you say it. Type arguments. mutating or consuming on a generic value method. Every argument: defaults aren't in the binary.

Throws, never guesses. A missing fact is an error, not a corrupted stack.

Hooking

```
Swift.Interceptor.attach(robot.greet, {
  onEnter(args) { console.log(this.self.name, args[0]); },
  onLeave(retval, error) { if (error !== undefined) console.log(error); },
});

// args by position; structs as objects, strings as strings,
// classes as facades, `any P` unwrapped, inout as a live view
Swift.Interceptor.attach(robot.$method("doubled"), {
  onEnter(args) { this.n = args[0]; },          // func doubled(_ n: inout Int)
  onLeave() { console.log(this.n.$fields); },    // what the callee wrote
});

for (const f of Swift.enumerateFunctions("*Robot.*"))
  Swift.Interceptor.attach(f, { onEnter(args) { console.log(f.name, args); } });
```

Frida saves every register. The bridge writes no code for a sync hook. It knows what each register means.

Decoded on demand. `args[i]`, `this.self`, `this.typeArguments`, the error: nothing is read until you touch it.

Any target with a signature. An address, `robot.greet`, a reflection member, a bound method, a `Swift.function`, an `enumerateFunctions` hit.

Refused when it can't plan. A generic signature it can't lower; a protocol-witness member (hook its address instead).

Properties and Subscripts

```
robot.badge;                // getter
robot.badge = "D2";         // setter
Robot.fleetSize = 13;       // static, through the type

grid.$subscript(1, 0, { labels: ["row", "column"] });
grid.$setSubscript(9, 1, "subscript(_:)"); // grid[1] = 9

// p.tracked += 3 goes through `modify`, a yield-once coroutine
const [modify] = Swift.enumerateFunctions("MyApp.Point.tracked.modify*");
Swift.Interceptor.attach(modify, {
    onEnter() { console.log("before", this.self.tracked); }, // 5
    onLeave(value) { console.log("after", value); }, // 8
});
```

Getters and setters are functions. Hook them like any other: `this.self` in both callbacks, the value as `retval` or `args[0]`.

`modify` is a **coroutine**. It yields the address, the caller mutates, then resumes it. `+=` and a nested write go here, not to the setter.

Hook a `modify`. `onLeave` fires when the caller is done, with the mutated value.

Not yet: `_read` accessors.

Closures

```
// Swift calls us: a JS function where Swift wants a closure
key.withUnsafeBytes(Swift.closure(buf => { bytes = buf.readBytes(); }));

source.$method("map", { typeArguments: [] })
    .call(7, Swift.closure(n => Number(n) * 6)); // 42

// We hook a function that takes one: the closure's two words
Swift.Interceptor.attach(key.$method("withUnsafeBytes"), {
    onEnter(args) { console.log(args[0].function, args[0].context); },
});
```

A JS function wherever Swift wants a closure.

The bridge builds both halves of a Swift closure, the function and its context.

Swift may keep it. The closure can escape and be stored; when Swift lets go of it, the JS side is freed too.

Works on arm64e. The function pointer is signed the way Swift expects for that closure type.

Shapes: scalars, String, (), a byte buffer.

Refused: closures as results, inout or consuming closures.

Async and Actors

```
const calc = Swift.type("MyApp.AsyncCalc").init(100);
await calc.addAsync(5);           // 105: a Promise
await calc.divideBaseBy(0);       // rejects with SwiftError

const session = Swift.type("MyApp.Session").init(); // actor Session
await session.advance();          // runs on the actor's executor

await Swift.function(app, "$s5MyApp12computeAsyncyS2iYaF").call(21);

Swift.Interceptor.attachAsync(fn, {
    onEnter(args, context) { ... }, // the async context
    onComplete(retval, error) { ... }, // the real completion
});
```

await an async method. It runs as a real Swift task; the promise resolves on completion and rejects on a throw.

Actors work. A call runs on the actor's executor, so calls serialise and state persists between them.

Hook to completion. onComplete fires with the real result or error, not at the first suspension.

Can't be fixed: a @MainActor call only completes if the target's main thread pumps its run loop.

What It Won't Guess

NOT IN THE BINARY

- type arguments of a generic call
- `mutating` / `consuming` on a generic value method
- every argument: defaults live in call sites
- `@frozen`, when the type's code gives no verdict: `Swift.markFrozen`

NOBODY CAN RECOVER

- code with no symbol: stripped and not exported
- code never emitted: inlined, specialised away
- closure captures whose type can't be resolved

REFUSED ON PURPOSE

- closures as results; `inout` or `consuming` closures
- tuples with an opaque element
- `_read` accessors, parameter packs, value generics
- a hook signature it can't plan

ABI uncertainty is never hidden behind a default that can corrupt memory:
derive the answer, pass it in a way that is correct either way, or throw.

Questions

Hooking Swift (the hard way)



Francesco Tamagni

[@mrmacete](#)

Security Research Engineer, NowSecure



Håvard Sørbo

[@hsorbo](#)

Security Research Engineer, NowSecure

Material github.com/mrmacete/hooks-swift-the-hard-way

Bridge github.com/hsorbo/frida-swift-bridge

Thank you!



MAScon Berlin
October 2026